

**Sunthareswaran
Sharujan**

TD01 & TD02 : Projet MarketPulse

Bootstrap Linux et traitement de données CSV / JSON en Python

Module MESIFI472326 – Python, Git, Linux · Groupe G07 · Équipe TSHA



Enseignant : ELANGA NKO'O Steve Patrick Lionel

Année Universitaire :2026/2027

Table des matières

1. Introduction.....	3
2. TD01 – Bootstrap + Linux.....	3
2.1. Contexte et objectifs.....	3
2.2. Structure du projet MarketPulse.....	3
2.3. Dépôt d'équipe et TEAM.md.....	4
2.4. Environnement de développement.....	4
2.5. Navigation Linux.....	4
2.5.1. Répertoire courant (pwd).....	4
2.5.2. Contenu du dépôt (ls, ls -la).....	5
2.5.3. Accès aux données (cd).....	5
2.6. Inspection des données de départ.....	6
2.6.1. Fichier JSON.....	6
2.6.2. Fichier CSV.....	7
2.7. Vérification de l'environnement.....	8
2.7.1. Python.....	8
2.7.2. Git.....	8
2.7.3. Exécution de MarketPulse.....	9
2.8. Validation du CORE.....	9
2.9. Questions optionnelles.....	10
2.9.1. Compter les observations.....	10
2.9.2. Autres affichages.....	10
2.9.3. Explorer tout le dépôt.....	10
2.9.4. Regarder l'état de Git.....	10
3. TD02 – Python + CSV / JSON.....	11
3.1. Contexte et objectifs.....	11
3.2. Partie 1 – Lecture du starter.....	11
3.3. Partie 2 – JSON et dictionnaires.....	11
3.4. Partie 3 – CSV et listes de dictionnaires.....	12
3.5. Partie 4 – Filtrage de l'instrument et du benchmark.....	14
3.6. Partie 5 – Premier et dernier cours.....	14
3.7. Partie 6 – Résumé réutilisable.....	15
3.8. Partie 7 – Une fonction main() lisible.....	17
3.9. Partie 8 – Sortie finale.....	18
3.10. Partie 9 – Récapitulatif des notions Python.....	18
3.11. Partie 10 – Passage au TD03.....	19
3.12. Code final.....	19
4. Synthèse : ce que j'ai appris.....	20

1. Introduction

Dans le module Python, Git, Linux (MESIFI472326), on construit au fil des séances un projet d'équipe appelé MarketPulse. Le contexte est celui d'une équipe data qui travaille pour un desk de trading, et l'application doit répondre à une question assez simple : comment se comporte un instrument financier par rapport à son indice de référence ?

Le projet va évoluer petit à petit. On commence avec des fichiers CSV et JSON en local, puis on passera à Yahoo Finance et Bloomberg, au calcul de performance, et enfin à une interface Dash. Pour l'instant, le starter commun compare l'action Apple (AAPL) au S&P 500, sur un mois de données quotidiennes.

Ce compte rendu regroupe les deux premiers TD. Le TD01 sert à mettre en place l'environnement : accéder au dépôt de l'équipe, se déplacer dans le projet avec le terminal, regarder les fichiers de données et lancer le programme. Le TD02 est la première vraie partie en Python : lire le JSON et le CSV, convertir les prix, séparer les deux séries et afficher un résumé avec une fonction réutilisable. Pour chaque partie, je montre les commandes que j'ai lancées avec des captures d'écran, puis j'explique les résultats et je réponds aux questions de l'énoncé.

2. TD01 – Bootstrap + Linux

2.1. Contexte et objectifs

Le besoin donné par le desk est : « *We need a common environment before we can work on MarketPulse* ». Dans ce TD on ne touche donc pas encore à la logique financière. Le but est de vérifier que tout le monde dans l'équipe peut récupérer le projet, le parcourir et l'exécuter. Concrètement, à la fin de la séance il faut savoir :

- identifier le projet et ses fichiers principaux ;
- accéder au dépôt de l'équipe ;
- utiliser un terminal et savoir dans quel dossier on se trouve ;
- lister et lire les fichiers du projet ;
- vérifier que Python et Git sont installés ;
- lancer le starter et repérer les fichiers d'entrée CSV et JSON.

2.2. Structure du projet MarketPulse

Le dépôt de départ (tawounfouet/esilv-marketpulse) contient un README.md, un TEAM_TEMPLATE.md, un CONTRIBUTING.md et les dossiers config/, data/, evidence/ et src/. Pour ce TD, voici les fichiers sur lesquels on se concentre :

Fichier	Rôle
README.md	Présentation du projet
TEAM_TEMPLATE.md	Modèle à copier pour créer TEAM.md
data/sample/instruments.json	Informations sur l'instrument et le benchmark
data/sample/prices.csv	Prix quotidiens des deux séries
src/main.py	Programme principal

2.3. Dépôt d'équipe et TEAM.md

On a créé un seul fork pour toute l'équipe (et pas un par personne). Le fork appartient au compte de Henri (@Henri127) et s'appelle `esilv-marketpulse-starter-g07-tSHA`. Henri nous a ensuite ajoutés comme collaborateurs, et chacun a vérifié qu'il avait bien accès au dépôt avec son propre compte GitHub.

Le fichier `TEAM.md` a été créé à partir de `TEAM_TEMPLATE.md`. Voici les informations qu'il contient :

Information	Valeur
Module	MESIFI472326 – Python, Git, Linux
Groupe de TD	G07
Équipe MarketPulse	TSHA
Membres (GitHub)	@Adirien, @sharujan0423-design (Sharujan Sunthareswaran), @Henri127
Dépôt de l'équipe	<code>esilv-marketpulse-starter-g07-tSHA</code>
Dépôt d'origine (upstream)	<code>tawounfouet/esilv-marketpulse</code>
Couple étudié	AAPL – Apple Inc. / S&P 500

Le fichier contient aussi les règles de travail de l'équipe : chacun utilise son propre compte GitHub et fait des commits identifiables, le travail sur les nouvelles fonctionnalités se fait dans des branches, on passera par des Pull Requests relues par un autre membre quand ce sera vu en TD, et on ne partage ni identifiants ni secrets dans le dépôt.

Comme le demande l'énoncé, il n'y a aucune donnée personnelle dans `TEAM.md` (pas d'e-mail, de numéro étudiant, de téléphone, de mot de passe ou de token). C'est logique : le dépôt est partagé et Git garde tout l'historique, donc une information mise par erreur resterait visible dans les anciens commits même si on la supprimait ensuite.

2.4. Environnement de développement

J'ai fait le TD01 dans le terminal de mon Mac (shell `zsh`, avec l'environnement `conda` « base » activé, c'est pour ça que le prompt commence par `(base)`). `macOS` est un système Unix, donc les commandes `pwd`, `ls`, `cd`, `cat`, `head` et `grep` marchent comme sous Linux. Pour le TD02, j'ai travaillé dans GitHub Codespaces, qui est l'environnement commun proposé par le cours : on a directement un terminal Linux, Python, Git et le dépôt dans le navigateur.

2.5. Navigation Linux

2.5.1. Répertoire courant (`pwd`)

`pwd` (print working directory) affiche le chemin complet du dossier dans lequel on se trouve. C'est important parce que toutes les commandes avec un chemin relatif, comme `python src/main.py`, partent de ce dossier.

```
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA % pwd
/Users/sharujansunthareswaran/esilv-marketpulse-starter-g07-tSHA
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA % █
```

Figure 1 – Commande `pwd` à la racine du projet.

On est bien à la racine du projet, dans le dossier esilv-marketpulse-starter-g07-tSHA de mon répertoire personnel.

2.5.2. Contenu du dépôt (ls, ls -la)

ls liste ce qu'il y a dans le dossier courant. Avec -l on a le détail de chaque élément (permissions, propriétaire, taille, date de modification) et avec -a on voit aussi les fichiers cachés, ceux dont le nom commence par un point.

```
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA % ls
ls -la
config          evidence        README.md      TEAM_TEMPLATE.md
CONTRIBUTING.md labs            requirements.txt
data            readiness      src
total 56
drwxr-xr-x@ 14 sharujansunthareswaran staff  448  7 oct.  17:49 .
drwxr-xr-x+ 115 sharujansunthareswaran staff 3680  7 oct.  17:49 ..
drwxr-xr-x  12 sharujansunthareswaran staff  384  7 oct.  18:00 .git
-rw-r--r--   1 sharujansunthareswaran staff  178  7 oct.  17:49 .gitignore
drwxr-xr-x   3 sharujansunthareswaran staff   96  7 oct.  17:49 config
-rw-r--r--   1 sharujansunthareswaran staff 2943  7 oct.  17:49 CONTRIBUTING.md
drwxr-xr-x   3 sharujansunthareswaran staff   96  7 oct.  17:49 data
drwxr-xr-x   3 sharujansunthareswaran staff   96  7 oct.  17:49 evidence
drwxr-xr-x   6 sharujansunthareswaran staff  192  7 oct.  17:49 labs
drwxr-xr-x   3 sharujansunthareswaran staff   96  7 oct.  17:49 readiness
-rw-r--r--   1 sharujansunthareswaran staff 10270  7 oct.  17:49 README.md
-rw-r--r--   1 sharujansunthareswaran staff  133  7 oct.  17:49 requirements.txt
drwxr-xr-x   3 sharujansunthareswaran staff   96  7 oct.  17:49 src
-rw-r--r--   1 sharujansunthareswaran staff  1446  7 oct.  17:49 TEAM_TEMPLATE.md
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA %
```

Figure 2 – Contenu du dépôt avec ls puis ls -la.

Le premier caractère de chaque ligne donne le type : d pour un dossier et - pour un fichier. . correspond au dossier courant, .. au dossier parent, et .git est le dossier caché où Git enregistre l'historique. J'ai retrouvé les éléments demandés :

Élément attendu	Type	Présent ?
README.md	Fichier	Oui
config	Dossier	Oui
data	Dossier	Oui
evidence	Dossier	Oui
src	Dossier	Oui
docs	Dossier	Non

Le dossier docs n'existe pas dans notre version du starter. À la place, il y a les dossiers labs et readiness et un fichier requirements.txt. Le dépôt a sûrement été mis à jour après l'écriture de l'énoncé, et ça ne pose pas de problème pour la suite.

2.5.3. Accès aux données (cd)

cd (change directory) sert à changer de dossier. Je suis descendu dans data puis dans sample, en vérifiant à chaque fois où j'étais avec pwd et ce qu'il y avait avec ls.

```

(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA % cd data
pwd
ls

cd sample
pwd
ls
/Users/sharujansunthareswaran/esilv-marketpulse-starter-g07-tSHA/data
sample
/Users/sharujansunthareswaran/esilv-marketpulse-starter-g07-tSHA/data/sample
bloomberg_reference_expected.json      instruments.json
bloomberg_reference_sample.json        prices.csv
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan sample % █

```

Figure 3 – Navigation dans data puis data/sample.

Dans data/sample on trouve bien les deux fichiers attendus, instruments.json et prices.csv. Il y a aussi deux fichiers bloomberg_reference_*.json qui ne servent pas dans ce TD ; ils seront sûrement utilisés plus tard pour la partie Bloomberg.

Pour revenir à la racine, on fait `cd ../..` : chaque `..` remonte d'un dossier, donc on passe de data/sample à data puis à la racine.

```

(base) sharujansunthareswaran@MacBook-Pro-de-sharujan sample % cd ../..
pwd
/Users/sharujansunthareswaran/esilv-marketpulse-starter-g07-tSHA
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-
g07-tSHA % █

```

Figure 4 – Retour à la racine avec `cd ../..`.

2.6. Inspection des données de départ

2.6.1. Fichier JSON

cat affiche tout le contenu d'un fichier texte :

```

/Users/sharujansunthareswaran/esilv-marketpulse-starter-g07-tSHA
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-
g07-tSHA % cat data/sample/instruments.json
{
  "instrument": {
    "ticker": "AAPL",
    "name": "Apple Inc.",
    "currency": "USD",
    "market": "NASDAQ"
  },
  "benchmark": {
    "ticker": "SP500",
    "name": "S&P 500",
    "currency": "USD",
    "market": "US"
  }
}
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-
g07-tSHA % █

```

Figure 5 – Contenu de data/sample/instruments.json.

Le fichier contient deux objets, instrument et benchmark, et chacun a les quatre mêmes champs : ticker, name, currency et market. Pour répondre aux questions :

- **Quel est l'instrument ?** Apple Inc., ticker AAPL, coté au NASDAQ en dollars.
- **Quel est le benchmark ?** L'indice S&P 500, ticker SP500, sur le marché américain.

- **Où sont stockés les noms et les tickers ?** Dans `data/sample/instruments.json`, dans les champs `name` et `ticker` des objets `instrument` et `benchmark`.

Le fait que les deux objets aient exactement la même structure va être utile : le programme pourra les traiter de la même manière. C'est ce qu'on fait au TD02 avec une seule fonction de résumé pour les deux.

2.6.2. Fichier CSV

`head` affiche les 10 premières lignes d'un fichier, ce qui suffit pour voir comment il est organisé sans tout afficher.

```
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-
g07-tSHA % head data/sample/prices.csv
date,ticker,open,high,low,close,volume
2026-09-01,AAPL,249.20,251.50,248.00,250.00,38000000
2026-09-01,SP500,6592.00,6615.00,6580.00,6600.00,0
2026-09-02,AAPL,250.80,252.80,249.60,251.30,39500000
2026-09-02,SP500,6607.00,6627.00,6595.00,6612.00,0
2026-09-03,AAPL,249.60,251.30,248.40,249.80,41000000
2026-09-03,SP500,6596.00,6613.00,6584.00,6598.00,0
2026-09-04,AAPL,251.30,253.60,250.10,252.10,42500000
2026-09-04,SP500,6612.00,6635.00,6600.00,6620.00,0
2026-09-08,AAPL,251.90,253.90,250.70,252.40,44000000
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-
g07-tSHA %
```

Figure 6 – Premières lignes de `data/sample/prices.csv`.

La première ligne est l'en-tête, avec les sept colonnes du fichier :

Colonne	Signification
date	Date de la séance (format AAAA-MM-JJ)
ticker	Série concernée : AAPL ou SP500
open	Prix à l'ouverture
high	Plus haut de la séance
low	Plus bas de la séance
close	Prix (ou niveau) à la clôture
volume	Nombre de titres échangés (0 pour l'indice, qui ne s'échange pas directement)

`grep` n'affiche que les lignes qui contiennent un motif. Avec `grep AAPL` et `grep SP500` on peut donc isoler chaque série :

```
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA % grep AAPL data/sample/prices.csv
grep SP500 data/sample/prices.csv
2026-09-01, AAPL, 249.20, 251.50, 248.00, 250.00, 38000000
2026-09-02, AAPL, 250.80, 252.80, 249.60, 251.30, 39500000
2026-09-03, AAPL, 249.60, 251.30, 248.40, 249.80, 41000000
2026-09-04, AAPL, 251.30, 253.60, 250.10, 252.10, 42500000
2026-09-08, AAPL, 251.90, 253.90, 250.70, 252.40, 44000000
2026-09-09, AAPL, 254.00, 255.70, 252.80, 254.20, 38000000
2026-09-10, AAPL, 254.30, 256.60, 253.10, 255.10, 39500000
2026-09-11, AAPL, 254.20, 256.20, 253.00, 254.70, 41000000
2026-09-14, AAPL, 256.60, 258.30, 255.40, 256.80, 42500000
2026-09-15, AAPL, 257.20, 259.50, 256.00, 258.00, 44000000
2026-09-16, AAPL, 256.70, 258.70, 255.50, 257.20, 38000000
2026-09-17, AAPL, 259.20, 260.90, 258.00, 259.40, 39500000
2026-09-18, AAPL, 259.30, 261.60, 258.10, 260.10, 41000000
2026-09-21, AAPL, 259.10, 261.10, 257.90, 259.60, 42500000
2026-09-22, AAPL, 261.10, 262.80, 259.90, 261.30, 44000000
2026-09-23, AAPL, 261.20, 263.50, 260.00, 262.00, 38000000
2026-09-24, AAPL, 263.00, 265.00, 261.80, 263.50, 39500000
2026-09-25, AAPL, 262.60, 264.30, 261.40, 262.80, 41000000
2026-09-28, AAPL, 263.30, 265.60, 262.10, 264.10, 42500000
2026-09-29, AAPL, 264.50, 266.50, 263.30, 265.00, 44000000
2026-09-30, AAPL, 266.00, 267.70, 264.80, 266.20, 38000000
2026-09-01, SP500, 6592.00, 6615.00, 6580.00, 6600.00, 0
2026-09-02, SP500, 6607.00, 6627.00, 6595.00, 6612.00, 0
2026-09-03, SP500, 6596.00, 6613.00, 6584.00, 6598.00, 0
2026-09-04, SP500, 6612.00, 6635.00, 6600.00, 6620.00, 0
2026-09-08, SP500, 6623.00, 6643.00, 6611.00, 6628.00, 0
2026-09-09, SP500, 6640.00, 6657.00, 6628.00, 6642.00, 0
2026-09-10, SP500, 6642.00, 6665.00, 6630.00, 6650.00, 0
2026-09-11, SP500, 6642.00, 6662.00, 6630.00, 6647.00, 0
2026-09-14, SP500, 6663.00, 6680.00, 6651.00, 6665.00, 0
2026-09-15, SP500, 6664.00, 6687.00, 6652.00, 6672.00, 0
2026-09-16, SP500, 6663.00, 6683.00, 6651.00, 6668.00, 0
2026-09-17, SP500, 6683.00, 6700.00, 6671.00, 6685.00, 0
2026-09-18, SP500, 6684.00, 6707.00, 6672.00, 6692.00, 0
2026-09-21, SP500, 6683.00, 6703.00, 6671.00, 6688.00, 0
2026-09-22, SP500, 6703.00, 6720.00, 6691.00, 6705.00, 0
2026-09-23, SP500, 6704.00, 6727.00, 6692.00, 6712.00, 0
2026-09-24, SP500, 6715.00, 6735.00, 6703.00, 6720.00, 0
2026-09-25, SP500, 6713.00, 6730.00, 6701.00, 6715.00, 0
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA %
```

Figure 7 – Séries AAPL et SP500 isolées avec grep (la fin de la série SP500 est coupée sur la capture).

On voit que les deux séries sont dans le même fichier : pour chaque date il y a une ligne AAPL et une ligne SP500, et c'est la colonne `ticker` qui permet de les différencier. Le CSV contient donc bien à la fois les observations de l'instrument et celles du benchmark.

En regardant les dates, j'ai remarqué que le 7 septembre 2026 n'apparaît pas. C'est le Labor Day, un jour férié où les marchés américains sont fermés. Septembre 2026 a 22 jours ouvrés, donc sans ce jour férié on a bien 21 séances, ce qui correspond aux 21 observations par série. AAPL passe de 250.00 le 1er septembre à 266.20 le 30 septembre.

2.7. Vérification de l'environnement

2.7.1. Python

```
(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA % python --version
Python 3.9.12
```

Figure 8 – Version de Python.

Python 3.9.12 est installé (il vient d'Anaconda, environnement « base »). C'est suffisant pour le projet, qui n'utilise que des modules de la bibliothèque standard : `csv`, `json` et `pathlib`.

2.7.2. Git

```
[(base) sharujansunthareswaran@MacBook-Pro-de-sharujan esilv-marketpulse-starter-g07-tSHA % git --version
git version 2.50.1 (Apple Git-155)
```

Figure 9 – Version de Git.

Git 2.50.1 est disponible. Comme demandé, je n'ai créé ni branche ni commit pendant ce TD, puisque le travail avec Git commence au TD03.

2.7.3. Exécution de MarketPulse

Depuis la racine du dépôt, j'ai lancé `python src/main.py`. La sortie est la même que celle attendue dans l'énoncé :

```
% python src/main.py
=== MarketPulse ===

Instrument
AAPL - Apple Inc.
Last price: 266.20 USD

Benchmark
SP500 - S&P 500
Last level: 6742.00

Period: 1 month
Interval: Daily

Observations
AAPL: 21
SP500: 21
```

Terminal

Cette exécution permet d'identifier chaque élément du projet :

Rôle	Élément
Programme	src/main.py
Métadonnées du marché	data/sample/instruments.json
Prix quotidiens	data/sample/prices.csv
Instrument	AAPL – Apple Inc. (dernier prix : 266.20 USD)
Benchmark	SP500 – S&P 500 (dernier niveau : 6742.00)

On remarque que le programme écrit « Last price » pour Apple mais « Last level » pour le S&P 500, et qu'il n'affiche pas de devise pour l'indice. C'est normal : un indice ne s'achète pas comme une action, c'est une valeur calculée exprimée en points.

2.8. Validation du CORE

Critère (definition of done)	Statut
Un dépôt d'équipe existe	Validé
TEAM.md est complété	Validé
Tous les membres ont accès au dépôt	Validé
Un environnement de développement fonctionne	Validé
Un terminal Linux/Unix fonctionne	Validé
Python est disponible (3.9.12)	Validé
Git est disponible (2.50.1)	Validé
MarketPulse s'exécute avec <code>python src/main.py</code>	Validé
Les fichiers CSV et JSON ont été inspectés	Validé

De mon côté, j'ai utilisé `pwd`, `ls`, `cd`, `cat`, `head` et `grep`, j'ai identifié le fichier JSON et le fichier CSV, vérifié les versions de Python et de Git, lancé le programme et reconnu l'instrument et le benchmark. Le CORE du TD01 est donc terminé.

2.9. Questions optionnelles

2.9.1. Compter les observations

`grep -c` compte le nombre de lignes qui contiennent le motif au lieu de les afficher. `grep -c AAPL` et `grep -c SP500` sur `prices.csv` donnent chacune 21, comme le compteur affiché par le programme.

Pourquoi c'est utile d'avoir deux séries alignées ? Pour comparer une action à son indice, il faut regarder leur évolution sur exactement les mêmes périodes. Comme les deux séries ont les mêmes 21 dates, la performance de chacune sur le mois (et plus tard les rendements quotidiens, la base 100 ou la performance relative) sera calculée sur la même fenêtre. S'il manquait une date dans une seule des deux séries, on comparerait par exemple un rendement d'Apple sur un jour avec un rendement de l'indice sur deux jours, et la comparaison serait faussée.

2.9.2. Autres affichages

- `ls -lh` : comme `ls -l`, mais les tailles sont écrites en K ou M au lieu d'octets, ce qui est plus lisible ;
- `head -n 5` : affiche seulement les 5 premières lignes au lieu de 10, donc l'en-tête et les 4 premières observations ;
- `tail -n 5` : fait l'inverse et affiche les 5 dernières lignes, c'est-à-dire les données les plus récentes, où l'on retrouve les dernières clôtures (266.20 pour AAPL et 6742.00 pour SP500).

2.9.3. Explorer tout le dépôt

`find . -type f` liste tous les fichiers du projet, sous-dossiers compris, et `grep -R "AAPL" .` cherche « AAPL » dans tous ces fichiers. On voit qu'on peut fouiller tout un projet depuis le terminal sans ouvrir les fichiers un par un.

2.9.4. Regarder l'état de Git

`git status` montre la branche sur laquelle on est et les fichiers modifiés depuis le dernier commit, et `git log --oneline` affiche l'historique des commits, un par ligne. Ces deux commandes ne font que lire l'état du dépôt, elles ne changent rien.

3. TD02 – Python + CSV / JSON

3.1. Contexte et objectifs

Le desk envoie toujours ses données sous forme de fichiers locaux : un JSON avec les infos sur les instruments et un CSV avec les prix quotidiens. MarketPulse doit lire ces deux fichiers, distinguer l'instrument du benchmark, garder les bonnes lignes pour chacun et afficher un résumé réutilisable. On ne calcule pas encore de rendement. Les objectifs du TD sont de savoir :

- reconnaître une liste et un dictionnaire en Python ;
- lire un JSON avec `json.load()` et un CSV avec `csv.DictReader` ;
- comprendre pourquoi les nombres d'un CSV arrivent sous forme de texte, et les convertir ;
- filtrer les observations par ticker avec une boucle et une condition ;
- écrire de petites fonctions réutilisables pour traiter l'instrument et le benchmark de la même façon.

3.2. Partie 1 – Lecture du starter

J'ai d'abord relancé le programme pour vérifier qu'il marchait toujours. En tapant directement `src/main.py`, j'ai eu l'erreur `zsh: permission denied` : le fichier n'a pas le droit d'exécution, donc le shell refuse de le lancer comme un programme. Il faut passer par l'interpréteur avec `python src/main.py`, et on retrouve la même sortie qu'au TD01.

En lisant `src/main.py`, j'ai repéré les éléments demandés :

Élément	Rôle
<code>import csv, import json</code>	Modules de la bibliothèque standard pour lire les deux formats
<code>Path</code> (module <code>pathlib</code>)	Gère les chemins de fichiers ; l'opérateur <code>/</code> sert à les assembler
<code>DATA_DIR</code>	<code>Path("data/sample")</code> , le dossier où sont les fichiers d'entrée
<code>load_instruments()</code>	Lit <code>instruments.json</code> avec <code>json.load()</code>
<code>load_prices()</code>	Lit <code>prices.csv</code> avec <code>csv.DictReader</code>
<code>filter_prices()</code>	Garde uniquement les lignes d'un ticker
<code>main()</code>	Lance et coordonne tout le programme

Réponses aux questions :

- **Quelle fonction lit le JSON ?** `load_instruments()`.
- **Quelle fonction lit le CSV ?** `load_prices()`.
- **Quelle fonction sélectionne un ticker ?** `filter_prices()`.
- **Quelle fonction coordonne l'exécution ?** `main()`.
- **Où sont les fichiers d'entrée ?** Dans `data/sample`, donné par la constante `DATA_DIR`. Comme c'est un chemin relatif, il faut lancer le programme depuis la racine du dépôt, sinon Python ne trouve pas les fichiers (`FileNotFoundError`).

3.3. Partie 2 – JSON et dictionnaires

Le JSON est chargé avec `json.load(file)`, qui transforme le contenu du fichier en objets Python. Pour voir ce qu'on obtient, j'ai ajouté quelques `print` temporaires à la fin de `main()` :

```

47     print()
48     print("Benchmark")
49     print(f'benchmark["ticker"] - {benchmark["name"]}')
50     print(f'Last level: {benchmark_latest["close"]}')
51     print()
52     print(f'Period: {LOOKBACK_LABEL}')
53     print(f'Interval: {INTERVAL_LABEL}')
54     print()
55     print("Observations")
56     print(f'instrument["ticker"]: {len(instrument_prices)}')
57     print(f'benchmark["ticker"]: {len(benchmark_prices)}')
58     print(type(instruments))
59     print(instruments.keys())
60
61
62 if __name__ == "__main__":
63     main()
64

```

```

=== MarketPulse ===
Benchmark
SP500 - S&P 500
Last level: 6742.00

Period: 1 month
Interval: Daily

Observations
AAPL: 21
SP500: 21
<class 'dict'>
dict_keys(['instrument', 'benchmark'])
@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $

```

Figure 10 – Type et clés de l'objet instruments.

`type(instruments)` donne `<class 'dict'>` et `instruments.keys()` donne `dict_keys(['instrument', 'benchmark'])`. Un objet JSON devient donc un dictionnaire en Python. J'ai ensuite regardé les dictionnaires à l'intérieur :

```

55     print("Observations")
56     print(f'instrument["ticker"]: {len(instrument_prices)}')
57     print(f'benchmark["ticker"]: {len(benchmark_prices)}')
58     print(type(instruments))
59     print(instruments.keys())
60     instrument = instruments["instrument"]
61     benchmark = instruments["benchmark"]
62
63     print(instrument["ticker"])
64     print(instrument["name"])
65     print(instrument["currency"])
66
67     print(benchmark["ticker"])
68     print(benchmark["name"])
69
70 if __name__ == "__main__":
71     main()
72

```

```

@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
Observations
AAPL: 21
SP500: 21
<class 'dict'>
dict_keys(['instrument', 'benchmark'])
AAPL
Apple Inc.
USD
SP500
@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $

```

Figure 11 – Accès aux champs de instrument et benchmark.

L'objet `instruments` est un dictionnaire qui contient deux objets métiers : `instrument`, qui décrit l'actif principal (AAPL, Apple Inc., USD), et `benchmark`, qui décrit l'indice de comparaison (SP500, S&P 500). Pour récupérer une information on enchaîne les clés, par exemple `instruments["instrument"]["ticker"]` vaut "AAPL". Une fois que c'était clair, j'ai enlevé ces print comme le demande l'énoncé.

Au passage, j'ai d'abord essayé de coller le `with open(...)` de l'énoncé directement dans le terminal, ce qui a donné l'erreur `zsh: unknown file attribute`. C'est normal : le terminal attend des commandes shell, pas du Python. Le code doit être écrit dans `src/main.py` puis lancé avec `python src/main.py`.

3.4. Partie 3 – CSV et listes de dictionnaires

`load_prices()` renvoie `list(csv.DictReader(file))`. `DictReader` prend la première ligne comme en-tête et transforme chaque ligne suivante en dictionnaire, avec les noms de colonnes comme clés. Voici ce que donne l'inspection :

```
src > main.py
29 def main():
56     print(f"instrument['ticker']: {len(instrument_prices)}")
57     print(f"benchmark['ticker']: {len(benchmark_prices)}")
58     instrument = instruments["instrument"]
59     benchmark = instruments["benchmark"]
60     prices = load_prices()
61
62     print(type(prices))
63     print(type(prices[0]))
64     print(prices[0])
65
66     if __name__ == "__main__":
67         main()
68
```

Problèmes Sortie Console de débogage **Terminal** Ports

Benchmark
SP500 - S&P 500
Last level: 6742.00
Period: 1 month
Interval: Daily
Observations
AAPL: 21
SP500: 21
class 'list'>

@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) \$ []

Figure 12 – Type de prices et de prices[0].

prices est une liste (`<class 'list'>`) et chaque élément, par exemple `prices[0]`, est un dictionnaire (`<class 'dict'>`) qui correspond à une ligne du CSV.

```
src > main.py
29 def main():
57     print(f"{'benchmark[''ticker'']}: {len(benchmark_prices)}")
58     instrument = instruments[instrument"]
59     benchmark = instruments["benchmark"]
60     prices = load_prices()
61
62     print(type(prices))
63     print(type(prices[0]))
64     print(prices[0])
65     print(prices[0]["close"])
66     print(type(prices[0]["close"]))
67     close = float(prices[0]["close"])
68
69 if __name__ == "__main__":
70     main()
71
```

Problèmes Sortie Console de débogage Terminal Ports

```
@sharujan0423-design → /workspaces/esily-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
Period: 1 month
Interval: Daily
Observations
AAPL: 21
SP500: 21
<class 'list'>
<class 'dict'>
{'date': '2026-09-01', 'ticker': 'AAPL', 'open': '249.20', 'high': '251.50', 'low': '248.00', 'close': '250.00', 'volume': '38000000'}
250.00
<class 'str'>
```

Figure 13 – Contenu de `prices[0]` et type de `close`.

Pourquoi les valeurs du CSV sont du texte ? Un CSV n'est qu'un fichier texte, il ne dit pas quel est le type de chaque colonne. Le module csv ne devine rien et renvoie tout sous forme de chaînes de caractères. On le voit sur la capture : `prices[0]["close"]` vaut `'250.00'` et son type est `<class 'str'>`. Il faut donc convertir avant de faire le moindre calcul :

```
close = float(prices[0]["close"])    # '250.00' (str) -> 250.0 (float)
```

Python

Sans conversion, les résultats seraient faux, voire impossibles : "250.00" + "1" donne "250.001" (les chaînes sont collées), une soustraction de deux chaînes provoque une `TypeError`, et une comparaison se fait dans l'ordre alphabétique, donc "9.5" > "10.2" est vrai. C'est pour ça que l'énoncé insiste sur la règle : texte CSV, conversion explicite, et seulement ensuite utilisation comme nombre.

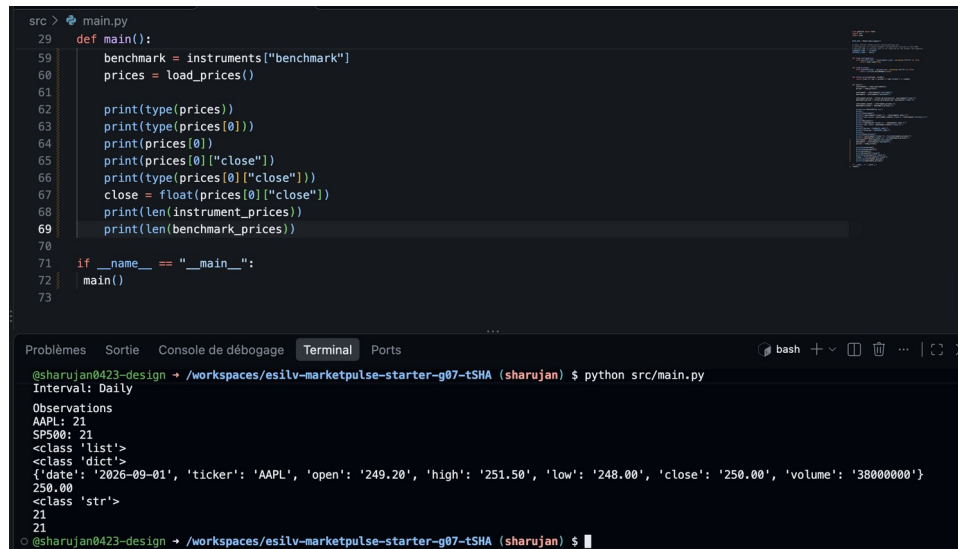
3.5. Partie 4 – Filtrage de l'instrument et du benchmark

Les deux séries sont mélangées dans le même fichier. `filter_prices()` les sépare avec une liste en compréhension, et je l'appelle une fois pour chaque objet, en utilisant le ticker lu dans le JSON plutôt que d'écrire « AAPL » en dur :

```
def filter_prices(prices, ticker):
    return [row for row in prices if row["ticker"] == ticker]

instrument_prices = filter_prices(prices, instrument["ticker"])
benchmark_prices = filter_prices(prices, benchmark["ticker"])
```

Python



The screenshot shows a code editor with a file named `main.py`. The code defines a `main()` function that loads prices, filters them by ticker, and prints the results. The terminal output shows the execution of `python src/main.py`, which prints the number of observations for AAPL (21) and SP500 (21), and then prints the first and last close prices for each.

```
src > main.py
29 def main():
59     benchmark = instruments["benchmark"]
60     prices = load_prices()
61
62     print(type(prices))
63     print(type(prices[0]))
64     print(prices[0])
65     print(prices[0]["close"])
66     print(type(prices[0]["close"]))
67     close = float(prices[0]["close"])
68     print(len(instrument_prices))
69     print(len(benchmark_prices))
70
71 if __name__ == "__main__":
72     main()
73
```

Problèmes Sortie Console de débogage Terminal Ports

```
@sharujan0423-design + /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
Interval: Daily
Observations
AAPL: 21
SP500: 21
<class 'list'>
<class 'dict'>
{'date': '2026-09-01', 'ticker': 'AAPL', 'open': '249.20', 'high': '251.50', 'low': '248.00', 'close': '250.00', 'volume': '38000000'}
250.00
<class 'str'>
21
21
```

Figure 14 – Nombre d'observations dans chaque série.

Les deux `len()` donnent 21, ce qui est le résultat attendu. La liste en compréhension fait exactement la même chose que cette boucle :

```
result = []
for row in prices:
    if row["ticker"] == ticker:
        result.append(row)
```

Python

Une boucle, une condition et un ajout : c'est le principe du filtrage. J'ai gardé la version en compréhension, qui est plus courte.

3.6. Partie 5 – Premier et dernier cours

`get_first_close` prend la première observation, lit le champ `close`, le convertit en `float` et renvoie le nombre. `get_last_close` fait pareil avec la dernière observation, grâce à l'indice `-1` qui donne le dernier élément d'une liste, quelle que soit sa taille :

```
def get_first_close(prices):
    return float(prices[0]["close"])

def get_last_close(prices):
    return float(prices[-1]["close"])
```

Python

```

src > main.py
29 def main():
30     # --- Affichage des premiers prix ---
31     print("Observations")
32     print(f"({instrument['ticker']}): {len(instrument_prices)}")
33     print(f"({benchmark['ticker']}): {len(benchmark_prices)}")
34     instrument = instruments[instrument]
35     benchmark = instruments[benchmark]
36     prices = load_prices()
37
38     print(type(prices))
39     print(type(prices[0]))
40     print(prices[0])
41     print(prices[0]["close"])
42     print(type(prices[0]["close"]))
43     close = float(prices[0]["close"])
44     print(len(instrument_prices))
45     print(len(benchmark_prices))
46     print(f"AAPL -> {get_first_close(instrument_prices):.2f}")
47     print(f"SP500 -> {get_first_close(benchmark_prices):.2f}")
48
49 def get_first_close(prices):
50     return float(prices[0]["close"])
51
52 if __name__ == "__main__":
53     main()

```

```

@sharujan0423-design ~ /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
Observations
AAPL: 21
SP500: 21
<class 'list'>
<class 'dict'>
{'date': '2026-09-01', 'ticker': 'AAPL', 'open': '249.20', 'high': '251.50', 'low': '248.00', 'close': '250.00', 'volume': '38000000'}
250.00
<class 'str'>
21
21
AAPL -> 250.00
SP500 -> 6600.00
@sharujan0423-design ~ /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $

```

Figure 15 – Test de `get_first_close` sur les deux séries.

```

src > main.py
35 def main():
36     # --- Affichage des premiers prix ---
37     print(type(prices[0]))
38     print(prices[0])
39     print(prices[0]["close"])
40     print(type(prices[0]["close"]))
41     close = float(prices[0]["close"])
42     print(len(instrument_prices))
43     print(len(benchmark_prices))
44     print(f"AAPL -> {get_first_close(instrument_prices):.2f}")
45     print(f"SP500 -> {get_first_close(benchmark_prices):.2f}")
46     print(f"AAPL -> {get_last_close(instrument_prices):.2f}")
47     print(f"SP500 -> {get_last_close(benchmark_prices):.2f}")
48     # --- Affichage des derniers prix ---
49     print(f"AAPL -> {get_last_close(instrument_prices):.2f}")
50     print(f"SP500 -> {get_last_close(benchmark_prices):.2f}")
51
52 def get_first_close(prices):
53     return float(prices[0]["close"])
54
55 if __name__ == "__main__":
56     main()

```

```

@sharujan0423-design ~ /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
<class 'dict'>
{'date': '2026-09-01', 'ticker': 'AAPL', 'open': '249.20', 'high': '251.50', 'low': '248.00', 'close': '250.00', 'volume': '38000000'}
250.00
<class 'str'>
21
21
AAPL -> 250.00
SP500 -> 6600.00
AAPL -> 266.20
SP500 -> 6742.00
AAPL -> 266.20
SP500 -> 6742.00
@sharujan0423-design ~ /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $

```

Figure 16 – Test de `get_last_close` sur les deux séries.

J'obtiens exactement les valeurs de l'énoncé. Comme le CSV est trié par date, `prices[0]` correspond au 1er septembre et `prices[-1]` au 30 septembre :

Série	Premier cours (01/09)	Dernier cours (30/09)
AAPL	250.00	266.20
SP500	6600.00	6742.00

3.7. Partie 6 – Résumé réutilisable

Au lieu d'écrire un affichage pour AAPL et un autre pour SP500, on fait une seule fonction `display_market_summary(asset, prices, show_currency=True)` qui reçoit l'actif (le

dictionnaire du JSON) et sa série de prix. Dans ma première version, j'appelais la fonction à l'intérieur d'un `print(...)` :

```
src > main.py
41 def main():
74     print(type(prices))
75     print(type(prices[0]))
76     print(prices[0])
77     print(prices[0]["close"])
78     print(type(prices[0]["close"]))
79     close = float(prices[0]["close"])
80     print(len(instrument_prices))
81     print(len(benchmark_prices))
82     print(f"AAPL -> {get_first_close(instrument_prices):.2f}")
83     print(f"SP500 -> {get_first_close(benchmark_prices):.2f}")
84     print(f"AAPL -> {get_last_close(instrument_prices):.2f}")
85     print(f"SP500 -> {get_last_close(benchmark_prices):.2f}")
86     # --- Affichage des derniers prix ---
87     print(f"AAPL -> {get_last_close(instrument_prices):.2f}")
88     print(f"SP500 -> {get_last_close(benchmark_prices):.2f}")
89     print(display_market_summary(instrument, instrument_prices))
90     print(display_market_summary(benchmark, benchmark_prices))
91
92
93 def get_first_close(prices):
94     return float(prices[0]["close"])
95
```

```
@sharujan0423-design + /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
AAPL -> 266.20
SP500 -> 6742.00
AAPL - Apple Inc.
Observations: 21
First close: 250.0
Last close: 266.2
None
SP500 - S&P 500
Observations: 21
First close: 6600.0
Last close: 6742.0
None
@sharujan0423-design + /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $
```

Figure 17 – Première version : un « None » s'affiche en trop.

Un None apparaît après chaque résumé. La fonction fait elle-même ses `print` mais n'a pas de `return`, donc elle renvoie None par défaut, et c'est ce None que le `print` extérieur affiche. Il suffit d'appeler la fonction directement, sans `print` autour.

```
src > main.py
20 def load_prices():
21     with open(WORKSPACE / "prices.csv", encoding="utf-8") as file:
22
23
24
25 def filter_prices(prices, ticker):
26     return [row for row in prices if row["ticker"] == ticker]
27
28 def get_last_close(prices):
29     return float(prices[-1]["close"])
30
31 def display_market_summary(asset, prices, show_currency=False):
32     print(f"{asset['ticker']} - {asset['name']}")
33     print(f"Observations: {len(prices)}")
34     print(f"First close: {float(prices[0]['close'])}")
35     print(f>Last close: {float(prices[-1]['close'])}")
36
37
38
39
40
41 def main():
42     instruments = load_instruments()
43     prices = load_prices()
44
```

```
@sharujan0423-design + /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
SP500 -> 6742.00
AAPL - Apple Inc.
Observations: 21
First close: 250.0
Last close: 266.2
None
SP500 - S&P 500
Observations: 21
First close: 6600.0
Last close: 6742.0
@sharujan0423-design + /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $
```

Figure 18 – Deuxième version : une seule décimale sur les prix.

Ensuite, les prix s'affichaient en 250.0 au lieu de 250.00. Un `float` ne garde pas les zéros inutiles du texte de départ (`float("250.00")` vaut 250.0), donc j'ai forcé deux décimales avec le format `:.2f` dans les f-strings.

Dans la version finale, la fonction appelle `get_first_close` et `get_last_close` au lieu de refaire la conversion elle-même, pour ne pas écrire deux fois la même chose. Le paramètre

show_currency permet d'afficher la devise pour Apple (en USD) mais pas pour l'indice (en points) :

```
def display_market_summary(asset, prices, show_currency=True):
    first_close = get_first_close(prices)
    last_close = get_last_close(prices)
    unit = f" {asset['currency']}" if show_currency else ""

    print(f"{asset['ticker']} - {asset['name']}")
    print(f"Observations : {len(prices)}")
    print(f"First close   : {first_close:.2f}{unit}")
    print(f>Last close    : {last_close:.2f}{unit}")
```

Python

C'est exactement l'idée de l'énoncé : même opération + données différentes = fonction réutilisable. Si on ajoutait un deuxième benchmark plus tard, il suffirait d'un appel en plus, sans réécrire l'affichage.

3.8. Partie 7 – Une fonction main() lisible

Avec tous les tests, ma fonction main() était devenue un peu chaotique : des print de test un peu partout, les données chargées deux fois, des affichages en double. Je l'ai réécrite pour qu'elle suive l'ordre donné dans l'énoncé : charger les métadonnées, charger les prix, récupérer l'instrument et le benchmark, filtrer les deux séries, afficher la configuration, puis les deux résumés.

```
46
47 def main():
48     # 1. Chargement des données
49     instruments = load_instruments()
50     prices = load_prices()
51
52     # 2. Sélection des entités
53     instrument = instruments["instrument"]
54     benchmark = instruments["benchmark"]
55
56     # 3. Filtrage des prix
57     instrument_prices = filter_prices(prices, instrument["ticker"])
58     benchmark_prices = filter_prices(prices, benchmark["ticker"])
59
60     # 4. Affichage final propre
61     print("=== MarketPulse ===")
62     print()
63
64     print("Market configuration")
65     print(f"Period   : {LOOKBACK_LABEL}")
66     print(f"Interval  : {INTERVAL_LABEL}")
67     print()
```

Figure 19 – main() réorganisée (début).

```
66     print(f"Interval : {INTERVAL_LABEL}")
67     print()
68
69     print("Instrument")
70     display_market_summary(instrument, instrument_prices, show_currency=True)
71     print()
72
73     print("Benchmark")
74     display_market_summary(benchmark, benchmark_prices, show_currency=False)
75     print()
```

Figure 20 – main() réorganisée (fin).

Maintenant `main()` ne fait plus aucun calcul ni mise en forme elle-même : elle appelle juste de petites fonctions qui ont chacune un rôle précis (lire, filtrer, extraire, afficher). Les fonctions sont définies au niveau du fichier, pas à l'intérieur de `main()`, et je n'ai pas changé l'architecture du projet, comme demandé.

3.9. Partie 8 – Sortie finale

```
@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $ python src/main.py
=== MarketPulse ===

Market configuration
Period : 1 month
Interval : Daily

Instrument
AAPL - Apple Inc.
Observations : 21
First close : 250.00 USD
Last close : 266.20 USD

Benchmark
SP500 - S&P 500
Observations : 21
First close : 6600.00
Last close : 6742.00
@sharujan0423-design → /workspaces/esilv-marketpulse-starter-g07-tSHA (sharujan) $
```

Figure 21 – Sortie finale de `python src/main.py`.

La sortie correspond exactement à celle attendue, y compris le format des nombres et l'alignement des libellés.

Comportement attendu	Statut
Instrument identifié (AAPL – Apple Inc.)	Validé
Benchmark identifié (SP500 – S&P 500)	Validé
Les deux séries sont filtrées	Validé
21 observations pour AAPL	Validé
21 observations pour SP500	Validé
Premier cours affiché	Validé
Dernier cours affiché	Validé
Conversion en nombre (float)	Validé
Résumé réutilisable	Validé

Je n'ai utilisé aucune bibliothèque externe (seulement `csv`, `json` et `pathlib`) et je n'ai ajouté aucun calcul de rendement.

3.10. Partie 9 – Récapitulatif des notions Python

Notion	Exemple dans MarketPulse	À quoi ça sert ici
Liste	<code>prices[0], prices[-1]</code>	Stocker les observations dans l'ordre
Dictionnaire	<code>instrument["ticker"]</code>	Stocker les infos d'un actif ou d'une ligne
Fonction	<code>def get_last_close(prices):</code>	Regrouper une logique qu'on réutilise
Condition	<code>if row["ticker"] == ticker:</code>	Garder les lignes d'un ticker
Boucle	<code>for row in prices:</code>	Parcourir toutes les observations
Conversion de type	<code>float(row["close"])</code>	Passer du texte du CSV à un nombre

3.11. Partie 10 – Passage au TD03

Comme demandé, je n'ai fait ni commit, ni branche en plus, ni Pull Request pendant ce TD. Les modifications de `src/main.py` restent dans l'arbre de travail du Codespace. Le TD03 partira de ces changements pour pratiquer Git en local : `git status` pour voir ce qui a été modifié, `git diff` pour regarder les changements en détail, `git add` et `git commit` pour les enregistrer, puis `git log` pour voir l'historique. Le premier vrai commit pourra donc correspondre au travail Python fait ici.

3.12. Code final

Voici la version finale complète de `src/main.py` :

```
from pathlib import Path
import csv
import json

DATA_DIR = Path("data/sample")

# These starter values mirror config/settings.yml.
# settings.yml is a human-readable configuration contract in the CORE.
# Parsing YAML is optional and is not required by the 18-hour lab sequence.
LOOKBACK_LABEL = "1 month"
INTERVAL_LABEL = "Daily"

def load_instruments():
    with open(DATA_DIR / "instruments.json", encoding="utf-8") as file:
        return json.load(file)

def load_prices():
    with open(DATA_DIR / "prices.csv", encoding="utf-8") as file:
        return list(csv.DictReader(file))

def filter_prices(prices, ticker):
    return [row for row in prices if row["ticker"] == ticker]

def get_first_close(prices):
    return float(prices[0]["close"])

def get_last_close(prices):
    return float(prices[-1]["close"])

def display_market_summary(asset, prices, show_currency=True):
    first_close = get_first_close(prices)
    last_close = get_last_close(prices)
    unit = f" {asset['currency']}" if show_currency else ""

    print(f"{asset['ticker']} - {asset['name']}")
    print(f"Observations : {len(prices)}")
    print(f"First close   : {first_close:.2f}{unit}")
    print(f>Last close    : {last_close:.2f}{unit}")

def main():
    # 1. Chargement des données
    instruments = load_instruments()
    prices = load_prices()
```

```

# 2. Sélection de l'instrument et du benchmark
instrument = instruments["instrument"]
benchmark = instruments["benchmark"]

# 3. Filtrage des deux séries
instrument_prices = filter_prices(prices, instrument["ticker"])
benchmark_prices = filter_prices(prices, benchmark["ticker"])

# 4. Affichage de la configuration puis des deux résumés
print("=== MarketPulse ===")
print()
print("Market configuration")
print(f"Period   : {LOOKBACK_LABEL}")
print(f"Interval  : {INTERVAL_LABEL}")
print()
print("Instrument")
display_market_summary(instrument, instrument_prices, show_currency=True)
print()
print("Benchmark")
display_market_summary(benchmark, benchmark_prices, show_currency=False)

if __name__ == "__main__":
    main()

```

Python

4. Synthèse : ce que j'ai appris

Ces deux séances posent les bases de MarketPulse. Avec le TD01, j'ai mis en place l'environnement et pris en main le terminal : savoir où on est avec `pwd`, explorer avec `ls` et `cd`, lire et filtrer des fichiers avec `cat`, `head` et `grep`, puis vérifier Python et Git. Rien qu'avec ces commandes, on comprend déjà bien les données avant d'écrire une ligne de code : un JSON qui décrit l'instrument et son benchmark, et un CSV qui contient leurs 21 observations quotidiennes, alignées sur les mêmes dates.

Le TD02 a permis de passer à Python. À la fin, le programme sort deux séries propres, alignées et converties en nombres, ce qui est exactement ce qu'il faudra pour les calculs de performance. Ce que je retiens surtout :

- **Faire attention aux types.** Un JSON devient un dictionnaire, un CSV lu avec `DictReader` devient une liste de dictionnaires, et toutes ses valeurs sont du texte qu'il faut convertir avant de calculer.
- **Le filtrage, c'est simple.** Une boucle, une condition et un ajout suffisent pour séparer deux séries rangées dans le même fichier.
- **Les fonctions évitent de se répéter.** Une seule fonction de résumé sert pour l'action et pour l'indice, et `main()` reste facile à lire.
- **Les erreurs aident à comprendre.** Le « permission denied », le code Python tapé dans le terminal, le `None` affiché ou les décimales manquantes m'ont chacun appris quelque chose sur le fonctionnement du terminal et de Python.